

# Web scraping

*Clase 11 - Cuando el dato vive en una página y nadie lo publicó en CSV*

# Hoy aprendemos a leer la web con código

## 1. HTML básico

Etiquetas, atributos y selectores  
CSS: el mapa de toda página.

## 2. rvest

`read_html()`, `html_elements()`,  
`html_text2()` y `html_table()`.

## 3. Ética y legalidad

`robots.txt`, términos de uso y  
tasas de solicitud con `Sys.sleep()`.

## 4. La era de la IA

Agentes que navegan, bloqueos  
anti-bot y licencias de contenido.

**Además: el equipo expositor presenta 'scraping en la era de la IA'.**

# Venimos de las fuentes ordenadas: datos abiertos

## El patrón data-raw/

La descarga es parte del análisis: script que baja el archivo, README con URL y fecha, limpieza aparte.

## download.file() y read\_csv()

Cuando hay URL directa a un CSV, dos funciones resuelven la ingesta completa.

## ¿Y si no hay CSV?

Muchos datos valiosos solo existen como páginas web. Hoy aprendemos a extraerlos con respeto y con código.

# El scraping es el plan C: antes van datos y APIs

## Plan A: datos abiertos

Si existe el CSV oficial, se descarga y listo (clase 10). Máxima confiabilidad, mínimo esfuerzo.

## Plan B: API oficial

Si el sitio ofrece una API, es la vía diseñada para máquinas: estable y documentada (clase 12).

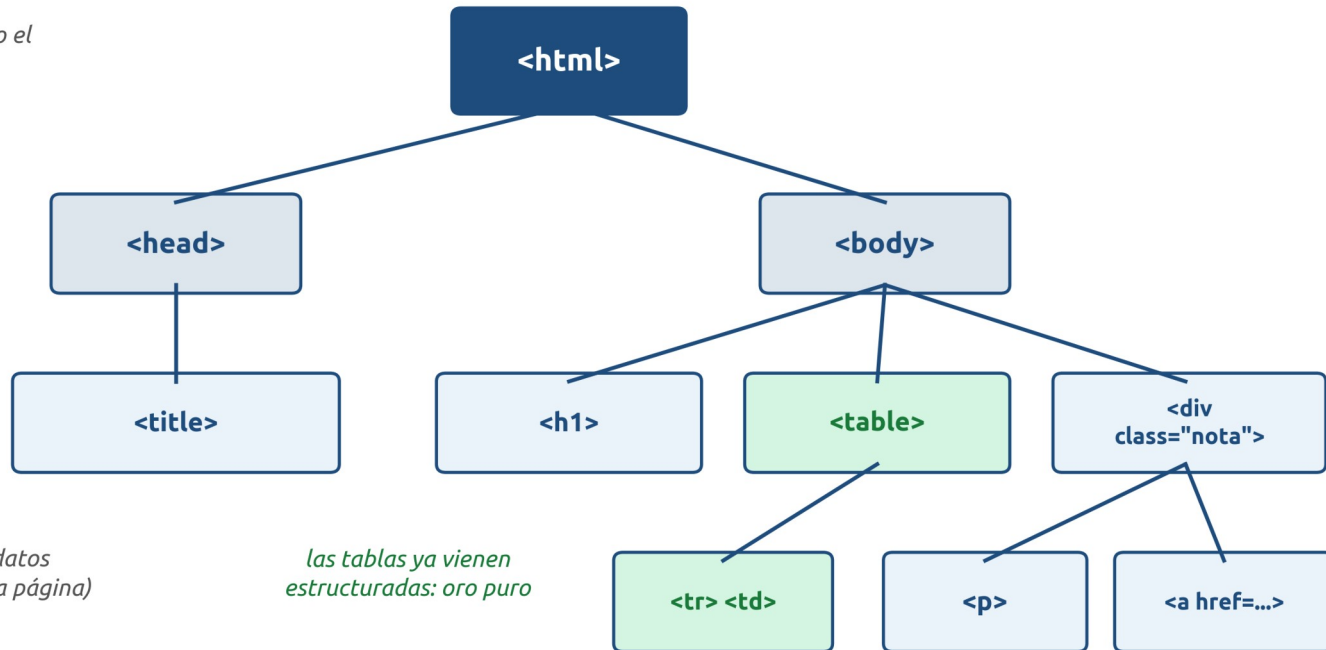
## Plan C: scraping

Solo cuando el dato existe únicamente como página web. Más frágil: si el sitio cambia su HTML, el script se rompe.

# Toda página web es un árbol de etiquetas HTML

Una página web es un árbol: el scraping navega sus ramas

*La raíz: todo el documento*



*<head>; metadatos  
(no se ven en la página)*

*las tablas ya vienen  
estructuradas: oro puro*

Para verlo en vivo: clic derecho en cualquier página, 'Inspeccionar'. Ese árbol es lo que scrapearemos.

# Etiquetas, atributos y clases: los ganchos

## Anatomía de un elemento

```
<a class="nota" href="https://ejemplo.mx">Leer más</a>
```

a = etiqueta | class y href = atributos | Leer más = texto

## Etiquetas frecuentes

h1...h6: títulos

p: párrafos

a: enlaces (href)

table, tr, td: tablas

div, span: contenedores

img: imágenes (src)

## Clases e identificadores

class agrupa elementos del mismo tipo visual (muchos por página).

id identifica un elemento único.

Son los ganchos favoritos del scraping.

# Los selectores CSS apuntan al dato que queremos

## Por etiqueta: "table"

Todas las tablas de la página. Así se extraen tablas de Wikipedia.

## Por clase: ".archive-item-date"

Todos los elementos con esa clase; el punto inicial es obligatorio. El caso típico: fechas o títulos de una lista de artículos.

## Por id: "#contenido"

El elemento único con ese identificador; se antecede con el símbolo de gato.

## Combinados: "div.nota a"

Los enlaces que viven dentro de un div con clase nota: se leen de izquierda a derecha, de padre a hijo.

# rvest: el paquete de scraping del tidyverse

## Qué es

Paquete de R inspirado en BeautifulSoup (Python), diseñado para encadenarse con %>%.

Se instala una vez: `install.packages("rvest")`.

## Sus cuatro verbos

`read_html()`: descarga el árbol

`html_elements()`: selecciona ramas

`html_text2()`: extrae el texto

`html_table()`: extrae tablas

## La lógica de siempre

Igual que con dplyr: pocas funciones que se combinan. Aprender rvest es aprender a escribir buenos selectores; el resto es el tidyverse que ya dominan.



# El flujo rvest: leer una vez, seleccionar, extraer

## Cinco pasos: del HTML crudo al tibble ordenado



`read_html()` se llama UNA vez y el resultado se guarda en un objeto. Probar selectores sobre ese objeto no vuelve a molestar al sitio.

# `read_html()` trae la página; `html_elements()` filtra

## Descargar y seleccionar

```
library(rvest)
```

```
pagina <- read_html("https://es.wikipedia.org/wiki/Nayarit")
```

```
titulos <- pagina %>%
```

```
  html_elements("h2")
```

## `html_elements()` acepta

Selectores CSS (etiqueta, .clase, #id) y también XPath para casos rebuscados. En singular, `html_element()` trae solo el primero.

## El resultado aún no es tabla

Devuelve nodos del árbol. Para volverlos datos usables faltan los extractores de la siguiente diapositiva.

# html\_text2() y html\_attr() extraen texto y enlaces

## Extraer contenido y atributos

```
titulos %>% html_text2()
# "Historia" "Geografía" "Economía" ...

pagina %>%
  html_elements("a") %>%
  html_attr("href") # los enlaces de la página
```

## ¿Por qué text2 y no text?

html\_text2() limpia espacios y saltos de línea como los ve el navegador; html\_text() entrega el texto crudo. Casi siempre conviene text2.

## html\_attr() para metadatos

Los enlaces (href) y las imágenes (src) viven en atributos, no en el texto. Así se scrapean listas de artículos con su URL.

# html\_table() convierte tablas HTML en tibbles

## El camino corto para tablas

```
tablas <- read_html(url_wiki) %>%  
  html_elements("table") %>%  
  html_table() # lista de tibbles  
  
poblacion <- tablas[[2]] %>% # la que nos interesa  
  janitor::clean_names()
```

## Idea clave

Si el dato ya está en una tabla HTML, `html_table()` hace todo el trabajo estructural: solo queda elegir la tabla correcta de la lista y limpiar nombres y tipos.

# robots.txt es una convención, no un candado

## Qué es

Archivo público en la raíz del sitio (ejemplo.mx/robots.txt) donde el dueño declara qué rutas pueden visitar los bots y cuáles no.

Se revisa ANTES de scrapear; leerlo toma un minuto.

## Qué NO es

No es un mecanismo técnico ni una ley: es una convención voluntaria.

En 2025 Cloudflare documentó que Perplexity accedía con rastreadores encubiertos a sitios que lo prohibían; perdió su estatus de bot verificado.

## Nuestra regla en el curso

Si robots.txt o los términos de uso dicen que no, es no. La reputación (y la calificación) no se juegan por una tabla.

# El semáforo ético del scraping

Tres niveles de riesgo que se evalúan antes de correr el script



**Adelante**

Hay datos abiertos o API oficial; robots.txt lo permite; el sitio es público y se scrapea despacio, identificándose



**Con cuidado**

HTML público sin API; términos de uso ambiguos; volumen alto: leer robots.txt, espaciar solicitudes, citar la fuente



**Alto**

Datos personales sin base legal; contenido tras contraseña o muro de pago; evadir bloqueos; ignorar un 'no' explícito del sitio

Ante la duda: buscar la fuente alternativa o pedir permiso al sitio. Casi siempre contestan.

# Scrapear despacio y con identificación honesta

## Pausas entre solicitudes

```
urls_paginas %>%  
  lapply(function(u) {  
    Sys.sleep(2)      # dos segundos de cortesía  
    read_html(u)  
  })
```

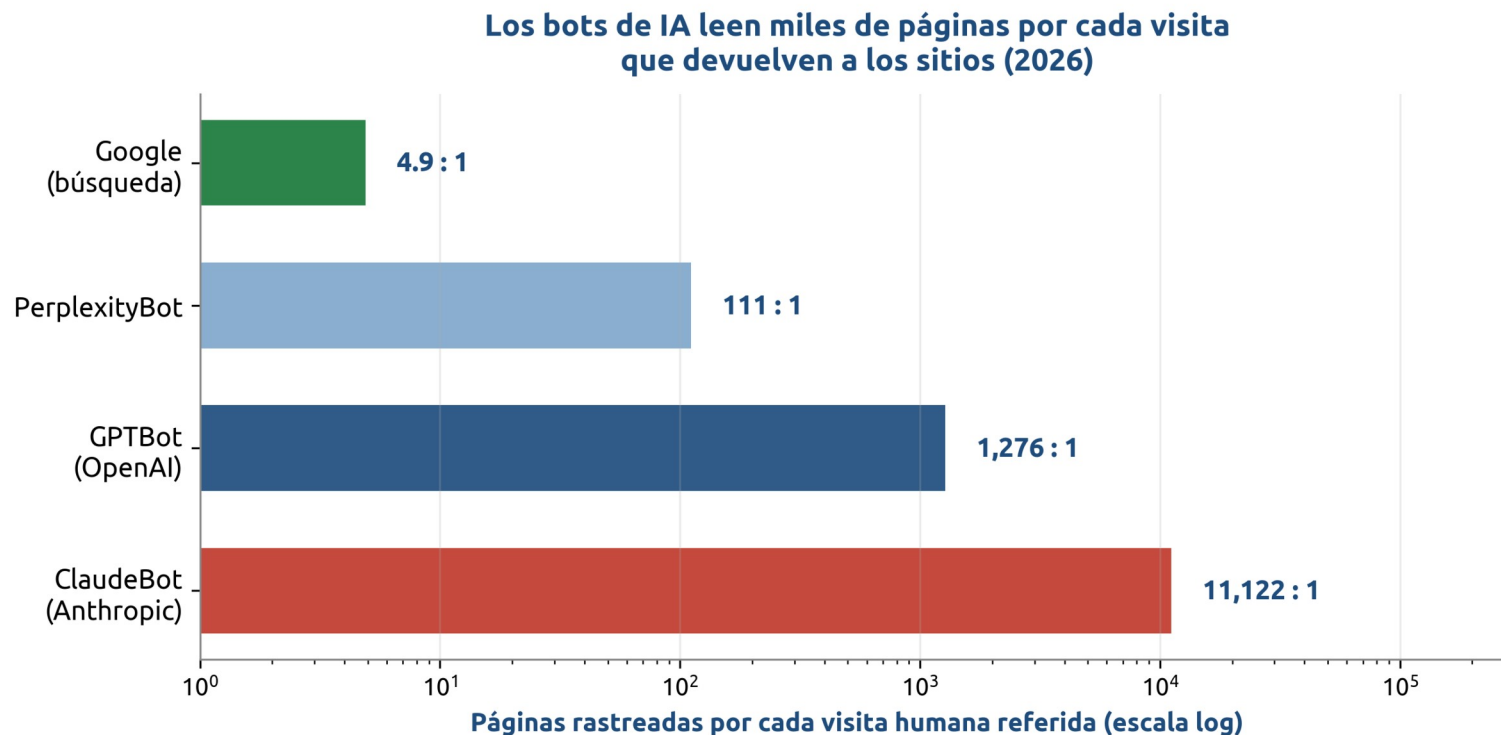
## Por qué importa

Cientos de solicitudes por segundo pueden tirar un sitio chico: eso ya no es recolección, es daño. Las pausas y los horarios valle son la diferencia.

## Identificarse

Un agente de usuario honesto con correo de contacto, no disfrazarse de navegador. Lección del caso hiQ-LinkedIn: lo público no es delito, pero el contrato del sitio sí obliga.

# Los bots ya generan más tráfico que los humanos



En junio de 2026 los bots generaron 57.5% del tráfico HTML (los humanos, 42.5%). El trato implícito de la web (te rastreo y te mando visitas) se rompió: los bots de IA leen miles de páginas y regresan casi nada.



# La web responde: bloqueo por defecto y peajes

## Bloqueo por defecto

Desde julio de 2025 Cloudflare bloquea a los rastreadores de IA salvo permiso explícito: el modelo pasó de opt-out a opt-in, con el respaldo de más de 50 medios y plataformas.

## Pago por rastreo (pay per crawl)

El sitio cobra por solicitud usando el código HTTP 402 (Payment Required); el bot declara cuánto está dispuesto a pagar. En beta desde 2025.

## Qué significa para ustedes

El scraping casual de sitios grandes será cada vez más difícil: más razón para preferir datos abiertos y APIs, y para scrapear solo donde son bienvenidos.

# Entrenar con contenido ajeno ya se litiga y licencia

## NYT contra OpenAI y Microsoft

Demanda activa desde 2023 por entrenar con millones de artículos; en 2026 sigue en etapa de pruebas, sin fallo de fondo.

## Acuerdo Anthropic - autores (2026)

1,500 millones de dólares por haber usado libros de bibliotecas piratas: el mayor acuerdo de derechos de autor en EE. UU. Entrenar con obras compradas sí fue declarado uso legítimo.

## La vía del contrato

AP, The Guardian, Washington Post y decenas más ya licencian su contenido a laboratorios de IA: el dato con valor se vende, no se regala.

# El equipo expositor: scraping en la era de la IA

*10 minutos + preguntas.*

## Tema de hoy: web scraping en la nueva realidad de la IA

El equipo expositor trabaja con el documento base del curso (carpeta 13: agentes que navegan, bloqueos anti-bot, licenciamiento) y al menos una fuente propia. El resto del grupo: una pregunta por equipo.

## Guía para escuchar

Mientras exponen, respondan en su cuaderno: si un agente de IA navega por ti, ¿quién es responsable de respetar robots.txt? ¿Cambia algo si el sitio cobra por rastreo?

# Ejercicio: scrapear una tabla real de Wikipedia

*En parejas, 25 minutos; requiere laptop con R y rvest instalado.*

## Instrucciones

1. Elijan un artículo de Wikipedia en español con una tabla que les interese (población por estado, medallero, elecciones...).
2. Revisen [es.wikipedia.org/robots.txt](https://es.wikipedia.org/robots.txt): ¿scrapear artículos está permitido?
3. Con `read_html()` + `html_elements("table")` + `html_table()`, extraigan la lista de tablas y localicen la suya.
4. Límpiela: `clean_names()`, tipos correctos con `mutate()` y un `count()` o `summarise()` que diga algo.
5. Documenten en comentarios: URL, fecha de scrapeo y número de tabla.

## Entregable (fin de la clase)

El script .R con el flujo completo y una captura del tibble final. Se sube al hilo de Canvas de la clase 11.

# Conclusiones de la clase 11

## Lo que se llevan hoy

1. Toda página es un árbol de etiquetas; los selectores CSS son la dirección de cada dato.
2. rvest resuelve el flujo completo: `read_html()` para leer, `html_elements()` para seleccionar, `html_text2()` y `html_table()` para extraer.
3. El scraping es el plan C: primero datos abiertos, luego APIs, al final HTML.
4. robots.txt y los términos de uso se respetan; se scrapea despacio (`Sys.sleep()`) y con identificación honesta.
5. La era de la IA endureció la web: bloqueos por defecto, pago por rastreo y litigios millonarios. Scrapear bien es también saber cuándo no scrapear.

# Recursos para profundizar

Documentación de rvest: [rvest.tidyverse.org](https://rvest.tidyverse.org) (incluye el tutorial 'SelectorGadget' para descubrir selectores sin leer HTML).

Wickham, H., Çetinkaya-Rundel, M. y Grolemund, G.: R for Data Science (2a ed.), capítulo de web scraping ([r4ds.hadley.nz](https://r4ds.hadley.nz)).

Documento del curso (Canvas): 'Web scraping en la nueva realidad de la IA', con las 15 fuentes verificadas de esta clase.

Para explorar HTML en vivo: el inspector del navegador (clic derecho, 'Inspeccionar') y [validator.w3.org](https://validator.w3.org).

Próxima clase: APIs con httr2, RAG y MCP; recuerden que el mini-proyecto 1 se entrega el 9 de septiembre.

# Gracias

**¿Preguntas?**